

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	ANALISIS Y DISEÑO DE ALGORITMOS				
TÍTULO DE LA PRÁCTICA:	Ordenamiento: Insertionsort, Selectionsort.				
NÚMERO DE PRÁCTICA:	2	AÑO LECTIVO:	2025-B	NRO. SEMESTRE:	IV (cuarto)
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	x	MÁXIMO DE ESTUDIANTES		
FECHA INICIO:	15/09/2025	FECHA FIN:	21/09/2025	DURACIÓN:	7 Días
RECURSOS A UTILIZAR: • Un computador. • Material del curso. • Bibliografía del curso					
DOCENTE(s): Diego Alonso Iquira Becerra					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Comprender el funcionamiento de los algoritmos de ordenamiento Insertion Sort y Selection Sort. • Analizar la eficiencia y complejidad de cada algoritmo en diferentes casos. • Aplicar los algoritmos en la resolución de problemas de clasificación. • Comparar su desempeño mediante métricas computacionales y teóricas.	
TEMAS: • Algoritmos de ordenamiento simples • Ordenamiento por selección (Selection Sort) • Ordenamiento por inserción (Insertion Sort)	
COMPETENCIAS	C.a Capacidad de análisis de problemas computacionales.
	C.b Diseño de soluciones algorítmicas eficientes.
	C.c Validación y verificación de algoritmos.
	C.d Comunicación técnica y redacción formal.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

1. Historia de los algoritmos de ordenamiento

Los algoritmos de ordenamiento forman parte de los fundamentos de la ciencia de la computación desde sus inicios. Durante las décadas de 1950 y 1960, con el surgimiento de los primeros lenguajes de programación y la necesidad de procesar grandes volúmenes de datos, surgieron numerosos algoritmos de ordenamiento que buscaban equilibrar simplicidad, eficiencia y uso de memoria.

Insertion Sort y Selection Sort son dos de los algoritmos de ordenamiento más antiguos y simples, comúnmente atribuidos a los trabajos tempranos en programación estructurada, particularmente en lenguajes como FORTRAN y ALGOL.

Aunque en aplicaciones modernas se prefieren algoritmos más eficientes.

2. Insertion Sort: Ordenamiento por inserción

Insertion Sort se inspira en la forma en que muchas personas ordenan cartas en la mano:

- Se empieza con una mano vacía y se van tomando cartas (elementos del arreglo) una a una.
- Cada nueva carta se inserta en la posición correcta con respecto a las que ya se tienen en la mano, desplazando hacia la derecha a las que sean mayores.

Descripción paso a paso:

1. Se asume que el primer elemento está ordenado.
2. Se toma el siguiente elemento (key) y se compara con los anteriores.
3. Si alguno de los anteriores es mayor que key, se lo mueve una posición a la derecha.
4. Cuando se encuentra el lugar adecuado, se inserta key.

Complejidad:

- Mejor caso: $\Omega(n)$ → La lista ya está ordenada; no se hacen movimientos.
- Peor caso: $O(n^2)$ → Lista en orden descendente; se mueven todos los elementos para cada inserción.
- Promedio: $\Theta(n^2)$

Ventajas:

- Simple de entender e implementar.
- Muy eficiente con listas pequeñas o casi ordenadas.
- Es estable: no cambia el orden de elementos iguales.
- Ordena en el lugar (in-place), sin usar memoria adicional.

Desventajas:

- Ineficiente con listas grandes desordenadas.
- Cada inserción puede requerir muchos desplazamientos.

3. Selection Sort: Ordenamiento por selección

Este algoritmo selecciona el elemento más pequeño (o más grande, según el orden deseado) de la lista y lo intercambia con el primer elemento no ordenado. Luego repite el proceso con el resto de la lista.

Descripción paso a paso:

1. Desde la posición 0 hasta $n-2$:
 - a. Buscar el mínimo elemento en el subarreglo desde la posición actual hasta el final.
 - b. Intercambiar ese mínimo con el elemento actual.
2. Repetir hasta que el arreglo esté ordenado.

Complejidad:

- Mejor caso: $\Omega(n^2)$ → Siempre hace $n(n-1)/2$ comparaciones, sin importar el orden.
- Peor caso: $O(n^2)$ → Igual que el mejor caso.
- Promedio: $\Theta(n^2)$

Ventajas:

- Simple de implementar.
- Menor número de movimientos comparado con Insertion Sort.
- Útil cuando los intercambios son costosos pero las comparaciones no.

Desventajas:

- No es estable: puede cambiar el orden relativo de elementos iguales.
- Siempre hace el mismo número de comparaciones, sin importar si el arreglo está ordenado.
- No es el algoritmo más eficiente para listas grandes.

4. Comparación

Criterio	Insertion Sort	Selection Sort
Enfoque	Inserta elementos ordenados	Selecciona mínimo y ubica
Estabilidad	Sí	No
In-place	Sí	Sí
Mejor caso (Ω)	$\Omega(n)$	$\Omega(n^2)$
Peor caso (O)	$O(n^2)$	$O(n^2)$
Promedio (Θ)	$\Theta(n^2)$	$\Theta(n^2)$
Comparaciones (en total)	Variable	Siempre $n(n-1)/2$
Intercambios	Muchos	Pocos
Eficiencia en listas	Muy buenas si casi ordenadas	Igual en todos los casos

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Problema: Dado un arreglo de n enteros, ordenarlo en orden ascendente utilizando ambos métodos e imprimir los pasos intermedios del algoritmo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 4

Descripción del problema:

```
int arr[] = {5, 2, 9, 1, 6};
```

Se debe ordenar este arreglo utilizando:

1. **Insertion Sort**
2. **Selection Sort**

En cada iteración, el programa debe imprimir el estado actual del arreglo para mostrar el progreso del algoritmo.

Insertion Sort

```
#include <iostream>
using namespace std;
```

```
void printArray(int arr[], int n, int paso) {
    cout << "Paso " << paso << ": ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

```
void insertionSort(int arr[], int n) {
    int paso = 0;
    printArray(arr, n, paso++);

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;

        printArray(arr, n, paso++);
    }
}
```

```
int main() {
    int arr[] = {5, 2, 9, 1, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Insertion Sort:\n";
    insertionSort(arr, n);

    return 0;
}
```

Selection Sort

```
#include <iostream>
using namespace std;

void printArray(int arr[], int n, int paso) {
    cout << "Paso " << paso << ": ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

void selectionSort(int arr[], int n) {
    int paso = 0;
    printArray(arr, n, paso++);

    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;

        swap(arr[i], arr[minIdx]);

        printArray(arr, n, paso++);
    }
}

int main() {
    int arr[] = {5, 2, 9, 1, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Selection Sort:\n";
    selectionSort(arr, n);

    return 0;
}
```

Algoritmo	Comparaciones (aprox.)	Intercambios	Observaciones
Insertion Sort	Variable	Alta	Realiza muchos desplazamientos
Selection Sort	Constante: $n(n-1)/2$	Baja	Menos movimientos, no es estable

- Insertion Sort es más eficiente si el arreglo ya está parcialmente ordenado.
- Selection Sort es consistente, pero no aprovecha el orden existente.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 6

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Ejercicio 1:

Implementa el algoritmo Insertion Sort para ordenar una lista de n nombres ingresados por el usuario.

Analiza su comportamiento con:

- Lista ya ordenada
- Lista en orden inverso
- Lista aleatoria

¿Cómo cambia el tiempo de ejecución?

Ejercicio 2:

Implementa el algoritmo Selection Sort sobre un arreglo de números flotantes. Muestra el número de comparaciones e intercambios realizados.

Ejercicio 3:

Crea una función que reciba un arreglo y un tipo de ordenamiento ("insertion" o "selection") y aplique el algoritmo correspondiente. Evalúa el rendimiento con 1000, 5000 y 10000 elementos aleatorios.

Ejercicio 4:

Desarrollen una aplicación en C++ que permita ordenar una lista de elementos sobre una temática libre elegida por el grupo (por ejemplo: productos, libros, canciones, deportistas, películas, videojuegos, etc.).

Cada elemento debe contener al menos los siguientes atributos:

- Un identificador o código (string o entero)
- Un nombre o título (string)
- Un valor numérico que servirá como criterio de ordenamiento (por ejemplo: precio, puntaje, año, duración, etc.)

Pasos a Seguir

1. Definir la temática y la estructura de los datos.
2. Cargar los datos dinámicamente desde un archivo .txt
3. Ordenar los elementos de mayor a menor según el valor numérico, usando:
 - a. **Insertion Sort**
 - b. **Selection Sort**

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

4. Implementar contadores para las comparaciones realizadas y los intercambios efectuados
5. Medir el tiempo de ejecución
6. Exportar la lista ordenada a un archivo .csv.

IV. CUESTIONARIO

1. ¿Cuál de los dos algoritmos es más eficiente si la lista ya está ordenada? ¿Por qué?
2. ¿Qué significa que un algoritmo sea "in-place"?
3. ¿Por qué se dice que Selection Sort no es estable?
4. ¿Cuál es la diferencia clave en el enfoque entre Insertion Sort y Selection Sort?
5. ¿Podrías optimizar alguno de los dos algoritmos para mejorar su rendimiento? ¿Cómo?

V. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- ✓ Cormen, T. et al. Introduction to Algorithms (MIT Press, 2022)
- ✓ Sedgewick, R. & Wayne, K. Algorithms (Addison-Wesley, 2011)
- ✓ Brassard, G. & Bratley, P. Fundamentals of Algorithmics (Prentice Hall, 1996)

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN				
TÉCNICAS: Ejercicios propuestos / Trabajos/ Preguntas formuladas / Proyectos / Resolución de casos		INSTRUMENTOS: Rubricas/Lista de cotejo		
CRITERIOS DE EVALUACIÓN				
Criterios	Muy Bueno	Bueno	Regular	Malo
Comprensión conceptual	Explica con claridad y precisión los conceptos, con ejemplos propios y completos.	Explica los conceptos de forma adecuada, con algunos ejemplos.	Explicación incompleta o con errores menores.	No comprende los conceptos o presenta errores graves.
Desarrollo de ejercicios	Implementa algoritmos correctos, justifica la corrección y compara la eficiencia.	Resuelve correctamente la mayoría de los ejercicios, con pocos errores.	Presenta errores importantes en la implementación o análisis.	No resuelve los ejercicios o están incorrectos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 8

Informe escrito	Presenta un informe ordenado, bien estructurado, con análisis y casos de prueba claros.	Informe claro pero con detalles faltantes en redacción o ejemplos.	Informe incompleto, desordenado o con análisis poco profundo.	No presenta informe o está ininteligible.
Participación y actitud	Participa activamente, colabora y cumple con plazos y responsabilidades.	Participa en la mayoría de actividades y cumple parcialmente.	Participa poco, con escasa colaboración o retrasos frecuentes.	No participa, incumple plazos y no colabora.